

Documentation Technique

Cartographie Interactive

Systeme de Gestion Collaborative des Consultations et Appels d'Offres



Architecture complete, guide de deployment et documentation des API

Version 1.0 | 2026

Service Commercial & Estimation des Coûts

Table des Matieres

1. Vue d'ensemble du projet

- 1.1 Objectif
- 1.2 Fonctionnalités principales
- 1.3 Stack technologique

2. Architecture du systeme

- 2.1 Architecture generale
- 2.2 Structure des dossiers
- 2.3 Flux de données

3. Base de données

- 3.1 Schema relationnel
- 3.2 Description des tables
- 3.3 Relations et contraintes

4. API et endpoints

- 4.1 Router auth
- 4.2 Router companies
- 4.3 Router projects (RBAC)
- 4.4 Router files

5. Systeme de permissions (RBAC)

- 5.1 Les 3 rôles
- 5.2 Logique de permission
- 5.3 Middleware d'autorisation

6. Guide de déploiement

- 6.1 Pre-requis
- 6.2 Installation pas à pas
- 6.3 Configuration des variables d'environnement
- 6.4 Commandes de maintenance

7. Guide de modification

- 7.1 Ajouter un dossier au processus
- 7.2 Modifier les permissions
- 7.3 Ajouter un endpoint API
- 7.4 Modifier le frontend

8. Perspectives d'évolution

1. Vue d'ensemble du projet

1.1 Objectif

La **Cartographie Interactive** est une application web collaborative destinee au service commercial et d'estimation des couts. Elle permet de :

- Visualiser les 14 etapes du processus commercial sous forme de flowchart interactif
- Organiser hierarchiquement les documents : Entreprise > Projet (AIM-XXXX) > Dossier par etape > Fichier
- Collaborer en equipe avec un systeme de roles et permissions granulaire (RBAC)
- Stocker et synchroniser les donnees dans le cloud via Firebase

1.2 Fonctionnalites principales

Fonctionnalite	Description
Flowchart interactif	14 etapes cliquables avec description, actions et documents attendus
Explorateur de fichiers	Navigation hierarchique Entreprise > Projet > Dossier > Fichier
Gestion des entreprises	CRUD complet, code couleur par entreprise
Gestion des projets	Code AIM-XXXX, statuts (active/won/lost/archived), notes
Upload de fichiers	Drag & drop par etape du processus
RBAC	3 roles : Commercial, Directeur, Admin avec permissions differenciees
Partage de projets	Proprietaire peut partager en mode viewer ou editor
Auth OAuth	Authentification via Kimi OAuth 2.0 avec JWT sessions
Synchronisation cloud	Firebase Firestore pour acces multi-appareils

1.3 Stack technologique

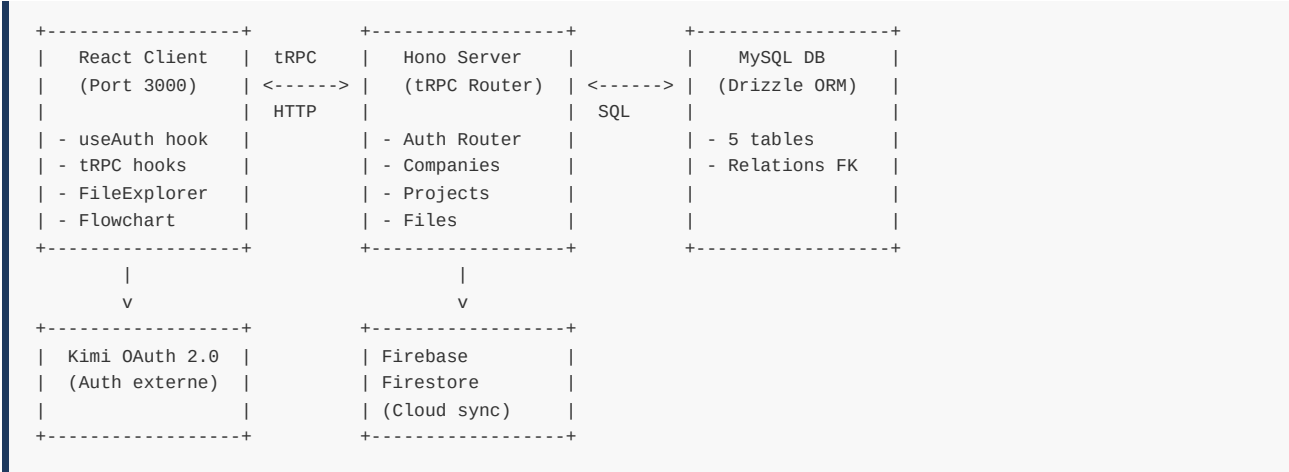
Couche	Technologie	Role
Frontend	React 19	Framework UI
	TypeScript	Type safety
	Tailwind CSS	Styling utilitaire
	shadcn/ui	Composants UI
Backend	Hono	Serveur HTTP
	tRPC 11	API type-safe
	Drizzle ORM	ORM MySQL
	Kimi OAuth	Authentification

Data	MySQL 8	Base de donnees relationnelle
	Firebase	Synchronisation cloud
Build	Vite	Build tool

2. Architecture du systeme

2.1 Architecture generale

L'architecture suit un pattern **fullstack moderne** avec separation claire entre frontend et backend :



2.2 Structure des dossiers

```

app/
├─ api/                                # Backend
│  ├─ router.ts                        # Routeur principal tRPC
│  ├─ middleware.ts                   # RBAC middleware
│  ├─ context.ts                      # Contexte tRPC (user, session)
│  ├─ auth-router.ts                 # Endpoints auth (me, signOut)
│  ├─ companies-router.ts            # CRUD entreprises
│  ├─ projects-router.ts             # CRUD projets + sharing
│  ├─ files-router.ts                # CRUD fichiers
│  ├─ queries/connection.ts          # Connexion Drizzle
│  └─ kimi/                          # SDK Kimi OAuth
├─ contracts/constants.ts             # Constantes partagees
├─ db/
│  ├─ schema.ts                      # Schema Drizzle (5 tables)
│  ├─ relations.ts                   # Relations Drizzle
│  └─ seed.ts                        # Donnees de demo
├─ src/
│  ├─ pages/                          # Pages React
│  │  ├─ Home.tsx                    # Page principale
│  │  └─ Login.tsx                   # Page de connexion
│  ├─ components/
│  │  ├─ FileExplorer/               # Explorateur de fichiers
│  │  │  ├─ CompanyList.tsx
│  │  │  ├─ ProjectList.tsx
│  │  │  ├─ StepFolderGrid.tsx
│  │  │  └─ StepFileList.tsx
│  │  ├─ Flowchart.tsx              # Flowchart interactif
│  │  └─ StepDetailPanel.tsx         # Panel de detail d'etape
│  ├─ components/ui/                 # Composants shadcn/ui
│  ├─ hooks/
│  │  ├─ useAuth.ts                  # Hook d'authentification
│  │  ├─ useCompanies.ts             # Hook tRPC companies
│  │  ├─ useProjects.ts              # Hook tRPC projects
│  │  └─ useFiles.ts                 # Hook tRPC files
│  ├─ data/flowData.ts               # Donnees du flowchart
│  ├─ types/index.ts                 # Types TypeScript
│  ├─ providers/trpc.tsx             # Provider tRPC client
│  ├─ App.tsx                        # Router principal
│  ├─ main.tsx                       # Point d'entree
│  └─ index.css                      # Styles globaux
├─ .env                             # Variables d'environnement
├─ package.json
├─ vite.config.ts
├─ drizzle.config.ts
└─ tsconfig.json

```

2.3 Flux de donnees

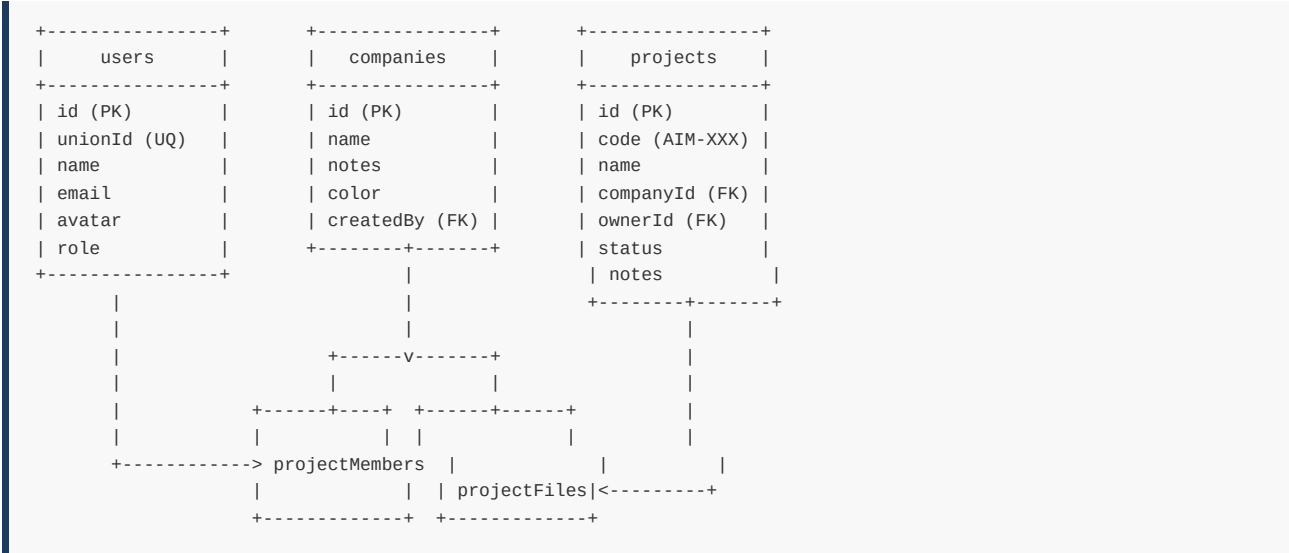
Le flux d'authentification et d'autorisation suit ces etapes :

1. **Login** : L'utilisateur clique sur "Se connecter avec Kimi" → redirection vers Kimi OAuth → callback avec code → echange contre JWT token → session creee
2. **Chaque requete** : Le client tRPC envoie le JWT dans les headers → le middleware verifie la session → injecte l'utilisateur dans le contexte
3. **Autorisation** : Le middleware RBAC verifie le role et les permissions projectMembers avant d'executer la requete

3. Base de donnees

3.1 Schema relationnel

La base de donnees contient 5 tables interconnectees :



3.2 Description des tables

Table : users

Stocke les utilisateurs authentifies via Kimi OAuth.

Colonne	Type	Contrainte	Description
id	serial	PK	ID auto-increment
unionId	varchar(255)	UQ, NOT NULL	ID unique Kimi
name	varchar(255)		Nom d'affichage
email	varchar(320)		Email
avatar	text		URL avatar
role	enum	NOT NULL, DEFAULT 'commercial'	commercial directeur admin
createdAt	timestamp	DEFAULT NOW()	Date de creation
updatedAt	timestamp		Date de mise a jour
lastSignInAt	timestamp		Derniere connexion

Table : companies

Stocke les entreprises clientes.

Colonne	Type	Contrainte	Description
id	serial	PK	ID auto-increment
name	varchar(255)	NOT NULL	Nom de l'entreprise
notes	text		Notes libres
color	varchar(7)	DEFAULT '#1E3A5F'	Couleur associee
createdBy	bigint	FK → users.id	Utilisateur createur
createdAt	timestamp	DEFAULT NOW()	Date de creation

Table : projects

Stocke les projets (consultations/appels d'offres) avec code AIM.

Colonne	Type	Contrainte	Description
id	serial	PK	ID auto-increment
code	varchar(50)	NOT NULL	Code AIM (ex: AIM-2024-001)
name	varchar(255)	NOT NULL	Nom du projet
companyId	bigint	FK → companies.id	Entreprise associee
ownerId	bigint	FK → users.id	Proprietaire du projet
status	enum	DEFAULT 'active'	active won lost archived
notes	text		Notes libres
createdAt	timestamp	DEFAULT NOW()	Date de creation

Table : projectFiles

Stocke les fichiers associes a un projet et une etape du processus.

Colonne	Type	Contrainte	Description
id	serial	PK	ID auto-increment
projectId	bigint	FK → projects.id	Projet associe
stepId	varchar(50)	NOT NULL	Etape (ex: 'reception', 'chiffrage')
name	varchar(255)	NOT NULL	Nom du fichier
fileType	enum	DEFAULT 'other'	pdf doc xls image other
size	varchar(50)		Taille affichee
date	varchar(10)		Date du fichier
createdBy	bigint	FK → users.id	Utilisateur createur
createdAt	timestamp	DEFAULT NOW()	Date de creation

Table : projectMembers

Table cle pour le RBAC. Lie les utilisateurs aux projets avec un niveau de permission.

Colonne	Type	Contrainte	Description
id	serial	PK	ID auto-increment
projectId	bigint	FK → projects.id	Projet
userId	bigint	FK → users.id	Utilisateur
permission	enum	DEFAULT 'viewer'	owner editor viewer pending
grantedBy	bigint	FK → users.id	Qui a donne la permission
createdAt	timestamp	DEFAULT NOW()	Date de creation

Conception RBAC : La table `projectMembers` est le coeur du controle d'accès. Chaque ligne represente un lien entre un utilisateur et un projet avec un niveau de permission specifique. Le propriétaire (owner) d'un projet est automatiquement ajoute a cette table lors de la creation.

3.3 Relations et contraintes

- **users 1-N companies :** Un utilisateur peut creer plusieurs entreprises (createdBy)
- **users 1-N projects :** Un utilisateur peut posseder plusieurs projets (ownerId)
- **companies 1-N projects :** Une entreprise a plusieurs projets (companyId)
- **projects 1-N projectFiles :** Un projet a plusieurs fichiers (projectId)
- **projects N-N users via projectMembers :** Un projet peut avoir plusieurs membres, un utilisateur peut acceder a plusieurs projets

4. API et Endpoints

L'API utilise **trRPC 11** qui fournit un typage end-to-end entre le frontend et le backend. Tous les endpoints sont definis dans des routeurs separes.

4.1 Router auth

Gere l'authentification et la session utilisateur.

Endpoint	Type	Auth	Description
auth.me	query	authedQuery	Retourne l'utilisateur connecte avec son role
auth.signOut	mutation	authedQuery	Deconnexion, suppression du cookie de session

4.2 Router companies

CRUD pour les entreprises. Creation ouverte a tous, modification restreinte aux directeurs/admins.

Endpoint	Type	Auth	Description
companies.list	query	authedQuery	Liste toutes les entreprises
companies.get	query	authedQuery	Detail d'une entreprise par ID
companies.create	mutation	authedQuery	Cree une entreprise (name, notes, color)
companies.update	mutation	directeurQuery	Modifie une entreprise (restreint)
companies.delete	mutation	directeurQuery	Supprime une entreprise (restreint)

4.3 Router projects (RBAC)

CRUD pour les projets avec verification des permissions a chaque operation.

Endpoint	Type	Auth	Description
projects.list	query	authedQuery + RBAC	Liste les projets visibles par l'utilisateur
projects.get	query	authedQuery + RBAC	Detail d'un projet avec permission de l'utilisateur
projects.create	mutation	authedQuery	Cree un projet, l'utilisateur devient owner
projects.update	mutation	authedQuery + canEdit	Modifie un projet (verifie permission editor)
projects.delete	mutation	authedQuery + owner/admin	Supprime un projet (owner ou admin uniquement)
projects.share	mutation	authedQuery + owner	Partage un projet avec un utilisateur (editor/viewer)
projects.revokeAccess	mutation	authedQuery + owner	Revoque l'acces d'un utilisateur a un projet
projects.listMembers	query	authedQuery + canView	Liste les membres d'un projet avec leurs permissions
projects.requestEdit	mutation	authedQuery + directeur	Directeur demande l'acces en edition (met pending)

4.4 Router files

CRUD pour les fichiers des projets.

Endpoint	Type	Auth	Description
files.list	query	authedQuery + canView	Liste les fichiers d'un projet (filtre par stepId optionnel)
files.get	query	authedQuery + canView	Detail d'un fichier par ID
files.create	mutation	authedQuery + canEdit	Ajoute un fichier a un projet (verifie permission editor)
files.delete	mutation	authedQuery + canEdit	Supprime un fichier (verifie permission editor)

5. Systeme de permissions (RBAC)

5.1 Les 3 roles

Le systeme implemente un **Role-Based Access Control** a 3 niveaux :

Commercial (role par default)

- **Visualiser** : Ses propres projets + les projets partages avec lui
- **Modifier** : Ses propres projets uniquement (lui est owner)
- **Partager** : Ses projets uniquement (en tant que owner)
- **Supprimer** : Ses projets uniquement

Directeur

- **Visualiser** : **Tous** les projets (vision globale)
- **Modifier** : Sur autorisation explicite du proprietaire (permission editor)
- **Partager** : Non
- **Supprimer** : Non

Admin

- **Tout** : Controle complet sur toutes les donnees
- Peut modifier, supprimer et partager n'importe quel projet
- Peut modifier les entreprises (update/delete)

5.2 Logique de permission

La logique est implementee dans `api/middleware.ts` via la fonction `getProjectPermission()` :

```
function getProjectPermission(userId, projectId, userRole):  
  SI userRole == "admin" :  
    RETOURNER "owner"          // Admin = full control  
  
  SI userRole == "directeur" :  
    member = chercher dans projectMembers  
      WHERE projectId = projectId AND userId = userId  
    SI member existe ET member.permission == "editor" :  
      RETOURNER "editor"  
    SINON :  
      RETOURNER "viewer"       // Directeur peut toujours voir  
  
  SI userRole == "commercial" :  
    project = chercher projet par projectId  
    SI project.ownerId == userId :  
      RETOURNER "owner"  
    member = chercher dans projectMembers  
      WHERE projectId = projectId AND userId = userId  
    SI member existe ET member.permission != "pending" :  
      RETOURNER member.permission  
    RETOURNER null             // Pas d'accès
```

5.3 Middleware d'autorisation

Quatre niveaux de middleware sont disponibles :

Middleware	Role requis	Usage
publicQuery	Aucun	Endpoints publics (ping)
authedQuery	Tout utilisateur connecte	Majorite des endpoints
directeurQuery	directeur OU admin	Modification entreprises
adminQuery	admin uniquement	Operations sensibles

Permission dynamique vs statique : Les middlewares statiques (authedQuery, directeurQuery, adminQuery) verifient le role. La permission sur un projet specifique est verifiee dynamiquement dans chaque endpoint via `getProjectPermission()` , `canEditProject()` et `canViewProject()` .

6. Guide de deployment

6.1 Pre-requis

Composant	Version minimum	Vérification
Node.js	20.x	<code>node --version</code>
npm	10.x	<code>npm --version</code>
MySQL	8.0	<code>mysql --version</code>
Git	2.x	<code>git --version</code>

6.2 Installation pas à pas

Étape 1 : Cloner et installer

```
git clone <repository-url> cartographie-app
cd cartographie-app
npm install
```

Étape 2 : Configurer la base de données

```
# Créer la base de données MySQL
mysql -u root -p
CREATE DATABASE cartographie CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
EXIT;
```

Étape 3 : Configurer les variables d'environnement

Le fichier `.env` est généré automatiquement par le script d'init. Vérifiez les valeurs suivantes :

```
DATABASE_URL=mysql://user:password@localhost:3306/cartographie
VITE_APP_ID=votre_app_id_portal
VITE_KIMI_AUTH_URL=https://your-auth-url
SESSION_SECRET=cle_secrete_pour_sessions
```

Étape 4 : Pousser le schéma

```
npm run db:push
```

Cette commande crée les 5 tables (users, companies, projects, projectFiles, projectMembers) automatiquement.

Étape 5 : Builder

```
npm run build
```

Etape 6 : Demarrer en production

```
npm start
```

L'application est accessible sur `http://localhost:3000`

6.3 Configuration des variables d'environnement

Variable	Description	Exemple
DATABASE_URL	URL de connexion MySQL	mysql://user:pass@localhost:3306/db
VITE_APP_ID	ID d'application dans le portail Kimi	app_123456789
VITE_KIMI_AUTH_URL	URL du service d'authentification Kimi	https://kimi.example.com
SESSION_SECRET	Cle secrete pour signer les JWT (32+ chars)	votre-cle-secrete-tres-longue
PORT	Port du serveur (optionnel, default 3000)	3000

6.4 Commandes de maintenance

Commande	Description
<code>npm run dev</code>	Mode developpement avec hot reload
<code>npm run build</code>	Build de production (frontend + backend)
<code>npm start</code>	Demarrage en mode production
<code>npm run check</code>	Vérification TypeScript (zero erreurs requis)
<code>npm run db:push</code>	Synchronise le schema avec la DB
<code>npm run db:generate</code>	Genere un fichier de migration SQL
<code>npm run db:migrate</code>	Applique les migrations en attente
<code>npm run format</code>	Formate le code avec Prettier

7. Guide de modification

7.1 Ajouter un dossier au processus commercial

Les dossiers du processus sont definis dans `src/data/flowData.ts`. Chaque etape a un champ `folderName` qui determine son dossier parent.

Etapes :

1. Ouvrir `src/data/flowData.ts`
2. Ajouter ou modifier un objet `FlowNode` dans le tableau `flowNodes`
3. Definir le `folderName` pour grouper les etapes par dossier
4. Si une nouvelle etape est ajoutee, ajoutez aussi son icone dans `Flowchart.tsx`

```
// Exemple : ajouter une nouvelle etape "Analyse concurrentielle"
{
  id: 'analyse-concu',
  step: 5,
  title: 'Analyse concurrentielle',
  type: 'step',
  folderName: '02 - DOSSIER ETUDE', // Groupe avec les etudes
  description: 'Etude des offres concurrentes...',
  details: ['Lister les concurrents', 'Analyser les prix', '...'],
  documentTypes: ['Benchmark concurrentiel'],
  icon: 'search'
}
```

Important : Le `folderName` est un simple texte. Les etapes avec le meme `folderName` sont automatiquement groupees dans l'Explorateur de Fichiers. Pas besoin de creer le dossier explicitement.

7.2 Modifier les permissions

Le RBAC est centralise dans `api/middleware.ts`.

Ajouter un nouveau role :

1. Modifier l'enum `role` dans `db/schema.ts`
2. Ajouter le role dans `requireRole()` dans `api/middleware.ts`
3. Mettre a jour la logique `getProjectPermission()` si necessaire
4. Executer `npm run db:push` pour mettre a jour la DB

Modifier les permissions d'un role existant :

Editez la fonction `getProjectPermission()` dans `api/middleware.ts`. Par exemple, pour donner aux Directeurs le droit de modifier sans autorisation :


```
// Avant (requiert autorisation)
if (userRole === "directeur") {
  const member = await db...
  if (member.length > 0 && member[0].permission === "editor") {
    return "editor";
  }
  return "viewer";
}

// Apres (modification libre)
if (userRole === "directeur") {
  return "editor"; // Directeur peut toujours editer
}
```

7.3 Ajouter un endpoint API

Pour ajouter un nouvel endpoint tRPC :

Etape 1 : Ajouter la logique dans le routeur

Dans `api/projects-router.ts` (ou le routeur approprié) :

```
myNewEndpoint: authedQuery
  .input(z.object({ projectId: z.number() }))
  .query(async ({ ctx, input }) => {
    // Verifier les permissions
    const hasAccess = await canViewProject(ctx.user.id, input.projectId, ctx.user.role);
    if (!hasAccess) throw new TRPCError({ code: "FORBIDDEN" });

    // Logique metier
    const result = await db.select().from(...)...;
    return result;
  }),
```

Etape 2 : Creer le hook frontend

Dans `src/hooks/useProjects.ts` :

```
export function useMyNewEndpoint(projectId: number) {
  return trpc.projects.myNewEndpoint.useQuery({ projectId });
}
```

Etape 3 : Utiliser dans un composant

```
const { data, isLoading } = useMyNewEndpoint(projectId);
```

7.4 Modifier le frontend

Ajouter une nouvelle page :

1. Creer le composant dans `src/pages/NouvellePage.tsx`
2. Ajouter la route dans `src/App.tsx`
3. Utiliser les hooks tRPC existants pour charger les donnees

Modifier les styles globaux :

Editez `src/index.css` . Les styles Tailwind sont déjà configurés. Vous pouvez ajouter des classes CSS personnalisées dans la section `@layer components` .

Ajouter un composant shadcn/ui :

```
npx shadcn add <component-name>  
# Exemple : npx shadcn add dialog
```

8. Perspectives d'evolution

Fonctionnalite	Description	Complexite
Notifications temps reel	Alertes via WebSocket pour demandes d'accès, modifications	Moyenne
Historique des modifications	Audit trail : qui a modifié quoi et quand, restauration de versions	Moyenne
Recherche full-text	Recherche dans noms de fichiers, notes, contenu des documents	Basse
Export PDF par projet	Rapport synthétique PDF automatique par projet	Basse
Signature électronique	Integration DocuSign/Signable pour validation des contrats	Haute
Application mobile	React Native avec partage de code pour accès terrain	Haute
Tableaux de bord	KPI commerciaux : taux de conversion, délais moyens	Moyenne
Import de données	Import CSV/Excel pour migration de données existantes	Basse

Architecture prête : Le système RBAC et l'API tRPC supportent toutes ces extensions naturellement. L'ajout de fonctionnalités ne nécessite pas de refonte de l'architecture existante.